FIAP — Análise e Desenvolvimento de Sistemas

ESGImpact

Plataforma ESG para Cidades Inteligentes

Migração de banco relacional para MongoDB (NoSQL)

5 collections · CRUD completo · Docker · Backend .NET 8

Disciplina: Bancos de Dados NoSQL

Setembro de 2026

Sumário

1. **Decisão: migrar um projeto existente**
   - 1.1 Por que NoSQL resolve melhor os desafios ESG
   - 1.2 Características do MongoDB efetivamente exploradas
2. **Visão geral da solução**
   - 2.1 Estrutura de pastas
   - 2.2 Infraestrutura Docker
3. **As 5 collections e sua relação com os pilares ESG**
   - 3.1 emission_factors — pilar Ambiental (E)
   - 3.2 simulations — pilar Ambiental (E)
   - 3.3 participants — pilar Social (S)
   - 3.4 offset_projects — pilares Ambiental e Social (E + S)
   - 3.5 governance_audits — pilar de Governança (G)
4. **Comandos de criação das collections e dos índices**
   - 4.1 createCollection
   - 4.2 Índices
   - 4.3 Carga inicial
5. **Operações CRUD realizadas**
   - 5.1 CREATE — insertOne e insertMany
   - 5.2 READ — find, projeções, arrays, geoespacial e explain
   - 5.3 UPDATE — $set, $inc, $push, $unset, upsert, arrayFilters e pipeline
   - 5.4 DELETE — deleteOne, deleteMany, findOneAndDelete e soft delete
6. **Aggregation Framework — o que era JOIN e GROUP BY**
7. **Backend .NET 8 migrado**
   - 7.1 Correspondência entre o modelo relacional e o de documentos
   - 7.2 MongoContext — o substituto do AppDbContext
   - 7.3 Como uma API tipada convive com esquema flexível
   - 7.4 Endpoints
8. **Como executar o projeto**
   - 8.1 Subir a infraestrutura
   - 8.2 Executar os scripts de CRUD
   - 8.3 Executar a API .NET
   - 8.4 Credenciais do ambiente
   - 8.5 Arquivos entregues
9. **Conclusão**

1. Decisão: migrar um projeto existente

A opção escolhida foi a MIGRAÇÃO. O projeto migrado é o ESGImpact, backend RESTful desenvolvido na fase anterior em ASP.NET Core 8 com Entity Framework Core e SQL Server, que calcula, classifica e armazena a pegada de carbono (kg CO_2) de atividades urbanas — deslocamentos, consumo de energia e alimentação — e oferece dashboard, motor de recomendações e cotação de compensação de carbono.

A migração não se limitou a trocar o driver de acesso a dados: o modelo foi redesenhado para o paradigma de documentos e ampliado de 3 tabelas para 5 collections, de modo a cobrir explicitamente os três pilares ESG — ambiental, social e de governança.

1.1 Por que NoSQL resolve melhor os desafios ESG

Três limitações concretas do modelo relacional anterior motivaram a decisão:

a) O dado ambiental não tem forma única

Um fator de emissão de transporte precisa de variantes por combustível (gasolina, etanol, elétrico); um fator de energia precisa do mix da matriz elétrica e de valores mensais; um fator de alimentação precisa de pegada hídrica e de mudança do uso da terra. Na tabela EmissionFactors isso significaria colunas nulas para a maioria das linhas, tabelas satélite com JOIN, ou JSON escondido em VARCHAR — os três caminhos ruins. No MongoDB cada documento carrega exatamente os blocos que fazem sentido para a sua categoria.

b) Cada projeto de compensação é tecnicamente diferente

Reflorestamento tem hectares, espécies e taxa de sobrevivência; biogás tem toneladas por dia e metros cúbicos de gás; solar tem kWp instalados e geração anual; mobilidade tem quilômetros de ciclovia. Modelar isso em SQL exigiria herança por tabela ou uma tabela larga majoritariamente vazia. Como documento, cada projeto declara o próprio bloco técnico (forestry, energy, waste, mobility, conservation) e ainda assim todos são consultáveis pelos campos comuns.

c) Governança é um fluxo de eventos heterogêneos e em evolução

Alteração de fator de emissão, revisão de política pública, acesso a dados sob a LGPD, incidente operacional e publicação de relatório são eventos com payloads completamente distintos. Pior: novos tipos de evento aparecem a cada nova exigência regulatória. Em Oracle ou SQL Server, cada novo tipo seria uma migração de esquema; no MongoDB é apenas um documento com um eventType novo.

1.2 Características do MongoDB efetivamente exploradas

Característica	Onde aparece neste projeto	Benefício ESG
Esquema flexível	Blocos variants, gridMix, seasonality, iotDevice, socialImpact, checklist	Absorve novas metodologias de cálculo e novos sensores sem migração nem downtime
Documentos polimórficos	participants guarda cidadão, organização e órgão público	Um só cadastro para atores sociais de naturezas diferentes
Subdocumentos e arrays	pricing[], goals[], variants[], checklist[], policies[]	Elimina tabelas de junção e reduz JOINs no caminho crítico
Desnormalização controlada	simulations.factorSnapshot	Auditabilidade: alterar o fator hoje não distorce o histórico de emissões
Aggregation Framework	$facet, $lookup, $unwind, $group nos serviços de analytics	Dashboard ESG completo em uma única ida ao banco

Característica	Onde aparece neste projeto	Benefício ESG
Índices geoespaciais 2dsphere	simulations.location (GeoJSON)	Análise de emissões por região da cidade — essencial em cidades inteligentes
Validação $jsonSchema (moderate)	emission_factors	Garante o núcleo obrigatório do dado sem engessar os campos extras
Escalabilidade horizontal	simulations é a coleção de maior volume e crescimento	Preparada para sharding por ownerCode/createdAt conforme a cidade cresce

2. Visão geral da solução

Item	Tecnologia
Banco de dados	MongoDB 7.0 (container Docker)
Interface de administração	Mongo Express 1.0.2 — http://localhost:8081
Database	esgimpact
Collections	5 (12 a 14 documentos cada)
Backend	.NET 8 / ASP.NET Core Web API + MongoDB.Driver 2.28
Autenticação	JWT Bearer (credenciais embutidas em participants)
Documentação da API	Swagger / OpenAPI
Orquestração	Docker Compose (rede, volumes, healthcheck, variáveis de ambiente)

2.1 Estrutura de pastas

```
ESGImpact-mongo/
├── docker-compose.yml            MongoDB + Mongo Express + API (perfil "api")
├── .env.example                  variáveis de ambiente
├── ESGImpact.Mongo.sln
├── README.md
├── mongo/
│   ├── init/                     executado automaticamente na 1ª subida do container
│   │   ├── 01-create-collections.js    createCollection, $jsonSchema e índices
│   │   ├── 02-seed-emission-factors.js
│   │   ├── 03-seed-participants.js
│   │   ├── 04-seed-offset-projects.js
│   │   ├── 05-seed-simulations.js
│   │   └── 06-seed-governance-audits.js
│   └── crud/                     roteiro de CRUD desta entrega
│       ├── 01-create.js          insertOne / insertMany nas 5 collections
│       ├── 02-read.js            find, projeção, array, $exists, geoespacial, explain
│       ├── 03-update.js          $set, $inc, $push, $unset, upsert, arrayFilters, pipeline
│       ├── 04-delete.js          deleteOne, deleteMany, findOneAndDelete, soft delete
│       └── 05-aggregations.js    $facet, $lookup, $unwind, $group
└── src/ESGImpact.Api/            backend .NET 8 migrado
    ├── Configuration/MongoSettings.cs
    ├── Data/MongoContext.cs              substitui o AppDbContext
    ├── Data/MongoBootstrapper.cs         substitui migrations + DbSeeder
    ├── Models/Entities.cs                5 entidades com [BsonExtraElements]
    ├── Repositories/DocumentRepository.cs CRUD genérico sobre BsonDocument
    ├── Services/                         negócio sobre Aggregation Framework
    ├── Controllers/                      CRUD das 5 collections + endpoints ESG
    └── ESGImpact.Api.http                roteiro completo de chamadas HTTP
```

2.2 Infraestrutura Docker

Todo o ambiente sobe com um único comando. Os scripts de mongo/init são montados em /docker-entrypoint-initdb.d e executados pelo próprio container na primeira inicialização, em ordem alfabética — de modo que criação de collections, índices e carga inicial acontecem sem qualquer passo manual.

docker-compose.yml (trecho)

```
services:
  esgimpact-mongo:
    image: mongo:7.0
    container_name: esgimpact-mongo
```

```
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER:-esgadmin}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:-Esg@Mongo2026}
      MONGO_INITDB_DATABASE: ${MONGO_DATABASE:-esgimpact}
    ports:
      - "${MONGO_PORT:-27017}:27017"
    volumes:
      - esgimpact-mongo-data:/data/db
      - esgimpact-mongo-config:/data/configdb
      - ./mongo/init:/docker-entrypoint-initdb.d:ro
      - ./mongo/crud:/scripts/crud:ro
    networks: [esgimpact-net]
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s

  esgimpact-mongo-express:
    image: mongo-express:1.0.2
    depends_on:
      esgimpact-mongo: { condition: service_healthy }
    ports: ["${ME_PORT:-8081}:8081"]

volumes:
  esgimpact-mongo-data:
  esgimpact-mongo-config:

networks:
  esgimpact-net: { driver: bridge }
```

Os comandos de execução estão detalhados na seção 8.

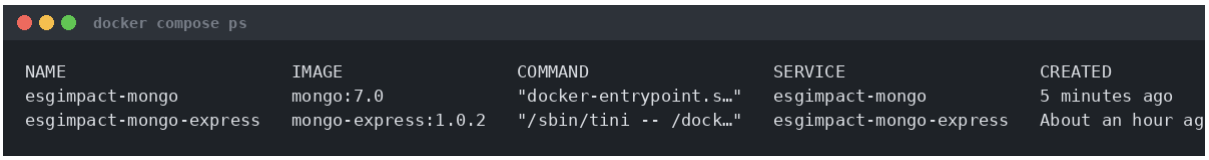


Figura 1 — Containers em execução (docker compose ps). Fonte: elaborado pelo autor.

3. As 5 collections e sua relação com os pilares ESG

#	Collection	Pilar	O que representa
1	emission_factors	E	Fatores de emissão de CO_2 por atividade, com metodologia, fonte e escopo GHG
2	simulations	E	Cada medição de pegada de carbono — app, importação de planilha ou sensor IoT
3	participants	S	Cidadãos, organizações e órgãos públicos: engajamento, metas, programas sociais e consentimento LGPD
4	offset_projects	E + S	Projetos de compensação: capacidade, preço por moeda, certificações e impacto social
5	governance_audits	G	Trilha append-only de auditoria, conformidade e rastreabilidade de decisões

3.1 emission_factors — pilar Ambiental (E)

Papel ESG: é a base metodológica de todo o cálculo ambiental. Guarda quanto CO_2 cada atividade emite por unidade, junto da fonte, do ano e da metodologia — informação que precisa ser auditável para que o inventário de emissões tenha valor de relatório ESG.

Documentos: 12 · **Variação de esquema:** transporte tem variants[] por combustível e regionalFactors[]; aviação tem haulFactors e fator de forçamento radiativo; energia tem gridMix e monthlyFactors[]; alimentação tem pegada hídrica; bicicleta é um documento propositalmente mínimo.

Três documentos da mesma collection, com formas diferentes

```
{
  code: 'CARRO',
  activity: 'Carro',
  factorKgCo2PerUnit: 0.21,
  unit: 'km',
  fieldLabel: 'Quilômetros percorridos',
  category: 'transporte',
  pillar: 'E',
  scope: 'escopo_3',
  isActive: true,
  source: { agency: 'GHG Protocol Brasil', year: 2025, methodology: 'tank-to-wheel' },
  variants: [                                  // <- bloco exclusivo de transporte
    { fuel: 'gasolina', factorKgCo2PerUnit: 0.21 },
    { fuel: 'etanol',   factorKgCo2PerUnit: 0.14 },
    { fuel: 'diesel',   factorKgCo2PerUnit: 0.25 },
    { fuel: 'eletrico', factorKgCo2PerUnit: 0.05 }
  ],
  updatedAt: ISODate('2026-09-01T00:00:00Z')
}

// MESMA collection, estrutura diferente:
{
  code: 'ENERGIA_ELETRICA',
  activity: 'Energia elétrica residencial',
  factorKgCo2PerUnit: 0.0385,
  unit: 'kWh',
  category: 'energia',
  scope: 'escopo_2',
  isActive: true,
  source: { agency: 'MCTI - Fator médio SIN', year: 2025, methodology: 'grid average' },
```

```
  gridMix: { hidrica: 0.58, eolica: 0.14, solar: 0.09, termica: 0.16, nuclear: 0.03 },
  monthlyFactors: [ { month: 1, factor: 0.041 }, { month: 2, factor: 0.040 }, ... ]
}

// E um documento mínimo, sem nenhum bloco extra:
{ code: 'BICICLETA', activity: 'Bicicleta', factorKgCo2PerUnit: 0.0,
  unit: 'km', category: 'transporte', pillar: 'E', isActive: true }
```

3.2 simulations — pilar Ambiental (E)

Papel ESG: é o registro factual das emissões. Cada documento é uma medição, e é sobre esta collection que o dashboard, o motor de recomendações e a cotação de compensação são calculados.

Documentos: 14 · **Variação de esquema:** leituras do app trazem location (GeoJSON) e context; leituras manuais são documentos enxutos; leituras de sensores trazem o bloco iotDevice com sensorId, janela de leitura e qualidade do sinal; importações corporativas trazem batch e fleet.

Duas decisões de modelagem merecem destaque. A primeira é a desnormalização controlada: o subdocumento factorSnapshot guarda o fator usado no instante do cálculo, de modo que uma alteração posterior do fator não altera o histórico já registrado e o dashboard não depende de JOIN. A segunda é o índice 2dsphere sobre location, que permite consultar emissões por região da cidade.

Três origens de dado convivendo na mesma collection

```
// leitura vinda do app, com geolocalização
{
  activityCode: 'CARRO', activity: 'Carro',
  value: 120, unit: 'km',
  estimatedKgCo2: 25.2, classification: 'Alto impacto',
  ownerCode: 'CID-003',
  createdAt: ISODate('2026-08-14T08:12:00Z'),
  factorSnapshot: { code: 'CARRO', factorKgCo2PerUnit: 0.21, unit: 'km',
                    capturedAt: ISODate('2026-08-14T08:12:00Z') },
  source: 'mobile',
  context: { channel: 'android', appVersion: '2.1.0' },
  location: { type: 'Point', coordinates: [-46.7220, -23.5680] },
  district: 'Butantã',
  tags: ['casa-trabalho']
}

// leitura vinda de um sensor IoT — mesma collection, bloco iotDevice
{
  activityCode: 'ENERGIA_ELETRICA', activity: 'Energia elétrica residencial',
  value: 402, unit: 'kWh', estimatedKgCo2: 15.48,
  ownerCode: 'ORG-001', createdAt: ISODate('2026-08-24T23:59:00Z'),
  source: 'iot',
  iotDevice: {
    sensorId: 'MED-SP-0471', model: 'SmartMeter X3', firmware: '4.2.1',
    readingWindow: { start: ISODate('2026-08-24T00:00:00Z'),
                     end:   ISODate('2026-08-24T23:59:00Z') },
    signalQuality: 0.93, anomalyFlag: 'pico_noturno'
  },
  site: { building: 'Sede TechVerde', floor: 3 }
}

// entrada manual — documento propositalmente enxuto
{ activityCode: 'CARNE_BOVINA', activity: 'Carne bovina', value: 1.2, unit: 'kg',
  estimatedKgCo2: 32.4, classification: 'Alto impacto', ownerCode: 'CID-004',
  createdAt: ISODate('2026-08-20T12:05:00Z'), source: 'manual' }
```

3.3 participants — pilar Social (S)

Papel ESG: representa o "S" de forma concreta: quem participa do programa, como se engaja, que metas assume, que programas sociais mantém e — ponto crítico de privacidade — qual consentimento LGPD concedeu.

Documentos: 12 · **Variação de esquema:** é a collection POLIMÓRFICA do projeto. O campo type define a forma: cidadao traz engagement, goals[] e acessibilidade; organizacao traz CNPJ, porte, corporateGoals[] e socialPrograms[]; orgao_publico traz esfera, policies[] e dados de frota. Alguns documentos ainda carregam o subdocumento credentials — os analistas que acessam o painel de governança.

No modelo de documentos, as três formas convivem em uma única collection e o login é um subdocumento embutido, protegido por um índice único e esparso em credentials.username — participantes sem credenciais não entram no índice.

Documentos polimórficos na collection participants

```
// cidadão COM credenciais de analista
{
  code: 'ANL-001', type: 'cidadao', name: 'Ana Lúcia Ferreira',
  district: 'Sé', city: 'São Paulo', role: 'Analyst',
  credentials: { username: 'analista',
                 passwordHash: '<PBKDF2-SHA256, 100.000 iterações>',
                 passwordSalt: '<base64>' },
  engagement: { level: 'ouro', points: 2480, streakDays: 41 },
  goals: [ { metric: 'kgCo2Mes', target: 60, achieved: true },
           { metric: 'simulacoesMes', target: 20, achieved: true } ],
  consent: { lgpdAccepted: true, shareAnonymizedData: true }
}

// organização — MESMA collection, outra forma
{
  code: 'ORG-001', type: 'organizacao', name: 'TechVerde Sistemas Ltda',
  cnpj: '12.345.678/0001-90', sector: 'tecnologia', size: 'media', employees: 240,
  corporateGoals: [ { metric: 'tonCo2Ano', target: 180, baseline: 260,
                      deadline: ISODate('2026-12-31T00:00:00Z') },
                    { metric: 'percentualFrotaEletrica', target: 40, baseline: 8 } ],
  esgReporting: { framework: 'GRI', lastReportYear: 2025, assuranceLevel: 'limitada' },
  consent: { lgpdAccepted: true, shareAnonymizedData: true }
}

// órgão público — terceira forma
{
  code: 'GOV-001', type: 'orgao_publico',
  name: 'Companhia de Transporte Metropolitano', sphere: 'estadual',
  policies: [ { code: 'CTM-EL-01',
                title: 'Eletrificação de 100% da frota até 2035',
                status: 'em_execucao' } ],
  fleet: { total: 1420, eletrica: 178, diesel: 1242 }
}
```

3.4 offset_projects — pilares Ambiental e Social (E + S)

Papel ESG: conecta o ambiental ao social. Cada projeto neutraliza emissões (E) e, ao mesmo tempo, declara o impacto social que gera (S): empregos criados, comunidades beneficiadas, famílias atendidas, alunos alcançados, catadores integrados. Os metas do ODS (sdgTargets) amarram o projeto à agenda global de sustentabilidade.

Documentos: 12 · **Variação de esquema:** cada tipo de projeto carrega o próprio bloco técnico — forestry (hectares, espécies), energy (kWp, geração), waste (ton/dia, captura de metano), mobility (km de ciclovia, veículos substituídos), conservation (hectares protegidos, blue carbon).

O preço da tonelada de carbono por moeda é armazenado no array pricing[] de cada projeto, como dado de negócio do próprio documento.

Projetos com blocos técnicos distintos

```
{
  code: 'PRJ-001',
  name: 'Reflorestamento Mata Atlântica - Vale do Ribeira',
  type: 'reflorestamento', status: 'ativo',
  capacityTonCo2Year: 1200,
  pricing: [ { currency: 'BRL', pricePerTon: 90.0 },     // preço por moeda
             { currency: 'USD', pricePerTon: 18.0 },
             { currency: 'EUR', pricePerTon: 16.5 } ],
  location: { type: 'Point', coordinates: [-47.8825, -24.4802],
              state: 'SP', city: 'Registro' },
  certifications: ['VCS', 'CCB'],
  forestry: { hectares: 340,                              // <- bloco de reflorestamento
              species: ['Ipê-amarelo', 'Jequitibá', 'Palmito-juçara'],
              survivalRate: 0.87 },
  socialImpact: { jobsCreated: 45, communitiesBenefited: 6, familiesInProgram: 120 },
  sdgTargets: [13, 15, 8]
}

// projeto de biogás — mesmo core, bloco técnico completamente diferente
{
  code: 'PRJ-007', name: 'Biodigestor de Resíduos Orgânicos - CEAGESP',
  type: 'biogas', status: 'ativo', capacityTonCo2Year: 1750,
  pricing: [ { currency: 'BRL', pricePerTon: 79.0 }, { currency: 'USD', pricePerTon: 16.0 } ],
  waste: { tonPerDay: 120, biogasM3PerDay: 9800, digestateUse: 'composto agrícola' },
  socialImpact: { catadoresIntegrated: 64, jobsCreated: 27 },
  sdgTargets: [11, 12, 13]
}
```

3.5 governance_audits — pilar de Governança (G)

Papel ESG: é o pilar de governança materializado. Sem trilha de auditoria, nenhum número ambiental tem credibilidade: é preciso saber quem alterou qual fator, quando, com que justificativa e com que aprovação. A collection é append-only por natureza e cobre também rastreabilidade de acesso a dados pessoais (LGPD), incidentes e publicação de relatórios.

Documentos: 12 · **Variação de esquema:** é a collection que mais se beneficia do esquema flexível — cada eventType tem um payload próprio e novos tipos entram sem qualquer migração.

eventType	Payload específico	Serve para
factor_updated	changes.before / changes.after / reason / approval	Rastrear alterações metodológicas
policy_review	policy + checklist[] com item, status, revisor e prazo	Revisão de políticas públicas e corporativas
data_access	query executada, documentsReturned, legalBasis	Conformidade LGPD de acesso
consent_change	consent.before / consent.after / channel / followUp	Trilha de consentimento do titular

eventType	Payload específico	Serve para
incident	severity, impact, resolution, closedAt	Gestão de incidentes e de qualidade do dado
report_published	period, metrics, assurance, publishedTo[]	Publicação auditada de indicadores

Três tipos de evento com payloads distintos

```
// alteração de fator de emissão, com antes/depois e aprovação
{
  eventType: 'factor_updated',
  occurredAt: ISODate('2026-07-18T09:41:00Z'),
  actor: { participantCode: 'ANL-002',
           name: 'Secretaria Municipal de Meio Ambiente', role: 'Analyst' },
  resource: { collection: 'emission_factors', documentCode: 'ENERGIA_ELETRICA' },
  changes: { before: { factorKgCo2PerUnit: 0.0426 },
             after:  { factorKgCo2PerUnit: 0.0385 },
             reason: 'Novo fator médio do SIN divulgado pelo MCTI' },
  compliance: { framework: 'GHG Protocol', status: 'conforme' },
  approval: { requiredBy: 'comite_esg', approvedBy: 'ANL-001',
              approvedAt: ISODate('2026-07-18T11:00:00Z') }
}

// revisão de política — payload totalmente diferente, mesma collection
{
  eventType: 'policy_review',
  occurredAt: ISODate('2026-07-25T13:00:00Z'),
  actor: { participantCode: 'GOV-001', role: 'Reviewer' },
  policy: { code: 'CTM-EL-01', title: 'Eletrificação de 100% da frota até 2035' },
  checklist: [
    { item: 'Meta quantificada e datada', status: 'ok', reviewer: 'ANL-002' },
    { item: 'Orçamento aprovado para 2026', status: 'ok', reviewer: 'ANL-002' },
    { item: 'Plano de infraestrutura de recarga', status: 'pendente',
      reviewer: 'ANL-001', dueAt: ISODate('2026-10-01T00:00:00Z') }
  ],
  compliance: { framework: 'GRI 305', status: 'parcial' }
}

// evento LGPD de consentimento
{
  eventType: 'consent_change',
  occurredAt: ISODate('2026-08-12T08:30:00Z'),
  actor: { participantCode: 'CID-005', name: 'Rafael Tavares', role: 'User' },
  resource: { collection: 'participants', documentCode: 'CID-005' },
  consent: { before: { shareAnonymizedData: true },
             after:  { shareAnonymizedData: false }, channel: 'app' },
  legalBasis: 'consentimento',
  compliance: { framework: 'LGPD', status: 'conforme' }
}
```

4. Comandos de criação das collections e dos índices

O script mongo/init/01-create-collections.js é executado automaticamente pelo container na primeira subida. Ele cria as cinco collections e todos os índices. A collection emission_factors recebe validação $jsonSchema com validationLevel "moderate": o núcleo do documento é obrigatório, mas campos adicionais permanecem livres — exatamente o equilíbrio que um dado ESG auditável precisa.

4.1 createCollection

01-create-collections.js — criação

```
db = db.getSiblingDB('esgimpact');

// 1) emission_factors (E) — com validação de esquema PERMISSIVA
db.createCollection('emission_factors', {
  validator: {
    $jsonSchema: {
      bsonType: 'object',
      required: ['code', 'activity', 'factorKgCo2PerUnit', 'unit', 'isActive'],
      properties: {
        code:               { bsonType: 'string', description: 'Código único da atividade' },
        activity:           { bsonType: 'string' },
        factorKgCo2PerUnit: { bsonType: 'number', minimum: 0 },
        unit:               { bsonType: 'string' },
        isActive:           { bsonType: 'bool' },
        pillar:             { enum: ['E', 'S', 'G'] }
      }
    }
  },
  validationLevel: 'moderate',
  validationAction: 'error'
});

// 2) simulations (E)
db.createCollection('simulations');

// 3) participants (S)
db.createCollection('participants');

// 4) offset_projects (E + S)
db.createCollection('offset_projects');

// 5) governance_audits (G)
db.createCollection('governance_audits');

print(db.getCollectionNames().sort());
```

4.2 Índices

01-create-collections.js — índices

```
// emission_factors
db.emission_factors.createIndex({ code: 1 }, { unique: true, name: 'ux_factor_code' });
db.emission_factors.createIndex({ category: 1, isActive: 1 }, { name: 'ix_factor_category_active'
});

// simulations
db.simulations.createIndex({ createdAt: -1 }, { name: 'ix_sim_createdAt' });
db.simulations.createIndex({ ownerCode: 1, createdAt: -1 }, { name: 'ix_sim_owner_createdAt' });
db.simulations.createIndex({ activityCode: 1 }, { name: 'ix_sim_activity' });
```

```
db.simulations.createIndex({ location: '2dsphere' }, { name: 'ix_sim_geo', sparse: true });

// participants
db.participants.createIndex({ code: 1 }, { unique: true, name: 'ux_participant_code' });
db.participants.createIndex({ 'credentials.username': 1 },
                            { unique: true, sparse: true, name: 'ux_participant_username' });
db.participants.createIndex({ type: 1, district: 1 }, { name: 'ix_participant_type_district' });

// offset_projects
db.offset_projects.createIndex({ code: 1 }, { unique: true, name: 'ux_project_code' });
db.offset_projects.createIndex({ status: 1, type: 1 }, { name: 'ix_project_status_type' });
db.offset_projects.createIndex({ 'pricing.currency': 1 }, { name: 'ix_project_currency' });

// governance_audits
db.governance_audits.createIndex({ occurredAt: -1 }, { name: 'ix_audit_occurredAt' });
db.governance_audits.createIndex({ eventType: 1, occurredAt: -1 }, { name: 'ix_audit_event' });
db.governance_audits.createIndex({ 'resource.collection': 1, 'resource.documentCode': 1 },
                                 { name: 'ix_audit_resource' });
```

Destaques: o índice esparso em credentials.username permite que apenas os participantes que possuem login entrem no índice único; o índice 2dsphere habilita consultas geoespaciais em simulations.location; e o índice composto (ownerCode, createdAt) cobre a consulta mais frequente do sistema — o histórico paginado de um participante.

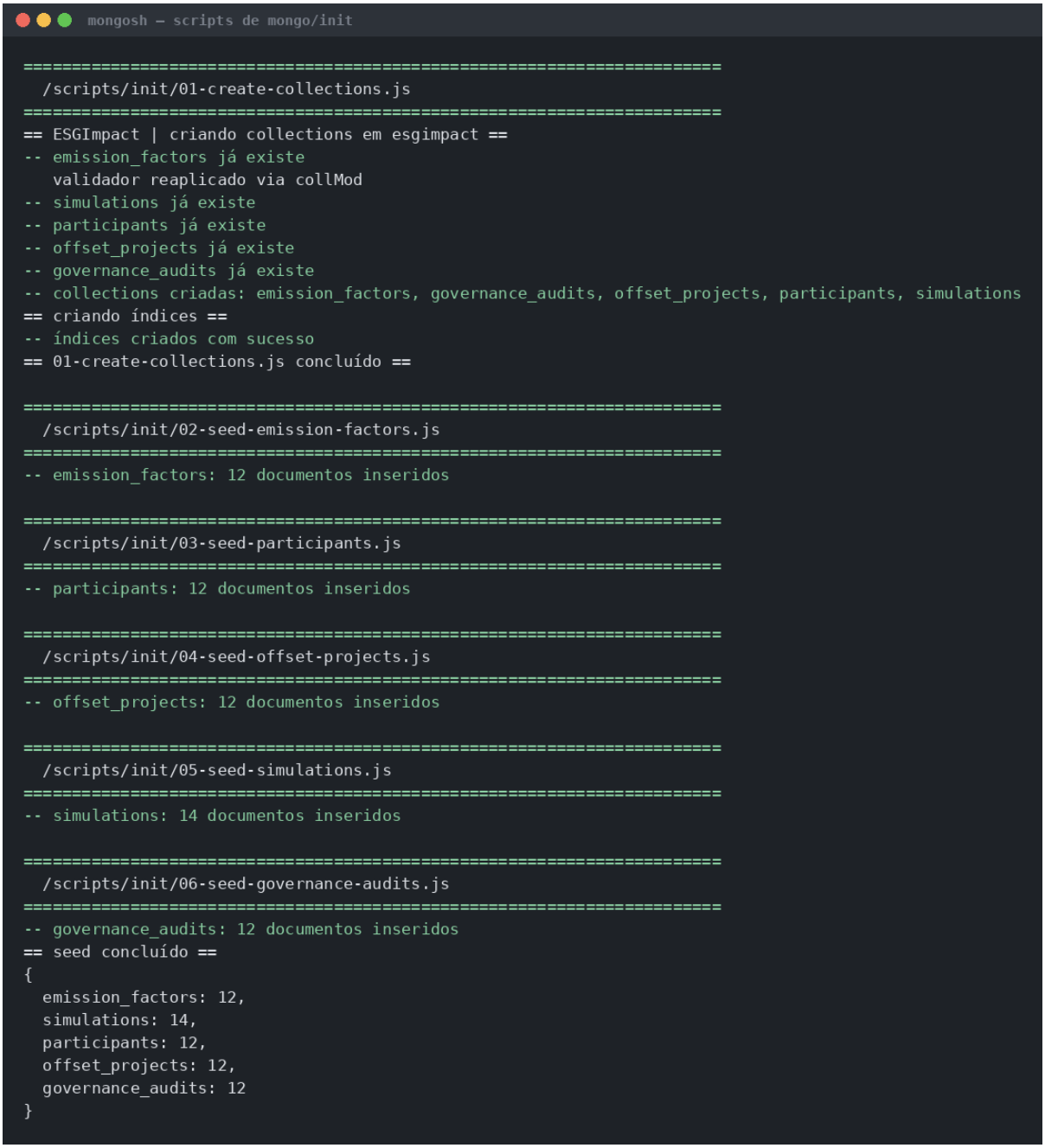


Figura 2 — Criação das collections e carga inicial. Fonte: elaborado pelo autor.

4.3 Carga inicial

Os scripts 02 a 06 de mongo/init carregam os dados. Cada collection ultrapassa o mínimo de 10 documentos exigido:

Collection	Documentos	Script de carga
emission_factors	12	02-seed-emission-factors.js
participants	12	03-seed-participants.js
offset_projects	12	04-seed-offset-projects.js
simulations	14	05-seed-simulations.js
governance_audits	12	06-seed-governance-audits.js
TOTAL	62	—

02-seed-emission-factors.js (trecho)

```
db.emission_factors.insertMany([
  { code: 'CARRO', activity: 'Carro', factorKgCo2PerUnit: 0.21, unit: 'km',
    fieldLabel: 'Quilômetros percorridos', category: 'transporte', pillar: 'E',
    scope: 'escopo_3', isActive: true,
    source: { agency: 'GHG Protocol Brasil', year: 2025, methodology: 'tank-to-wheel' },
    variants: [ { fuel: 'gasolina', factorKgCo2PerUnit: 0.21 },
                { fuel: 'etanol',   factorKgCo2PerUnit: 0.14 },
                { fuel: 'diesel',   factorKgCo2PerUnit: 0.25 },
                { fuel: 'eletrico', factorKgCo2PerUnit: 0.05 } ],
    updatedAt: now },
  /* ... 11 outros fatores: AVIAO, ONIBUS, METRO, BICICLETA, MOTO, CARNE_BOVINA,
         FRANGO, VEGETAL, ENERGIA_ELETRICA, GAS_ENCANADO, RESIDUO_ATERRO ... */
]);

print('-- emission_factors: ' + db.emission_factors.countDocuments() + ' documentos inseridos');
```

O script de simulações é o mais interessante da carga: em vez de valores fixos, ele LÊ o fator na collection emission_factors, calcula a emissão, classifica o impacto e grava o factorSnapshot — garantindo que os dados semeados sejam internamente consistentes.

05-seed-simulations.js (trecho)

```
function build(doc) {
  const f = db.emission_factors.findOne({ code: doc.activityCode });
  if (!f) throw new Error('Fator não encontrado: ' + doc.activityCode);

  const kg = Math.round(doc.value * f.factorKgCo2PerUnit * 100) / 100;

  return Object.assign({
    activityCode: f.code, activity: f.activity,
    value: doc.value, unit: f.unit,
    estimatedKgCo2: kg, classification: classify(kg),
    ownerCode: doc.ownerCode, createdAt: doc.createdAt,
    factorSnapshot: { code: f.code, factorKgCo2PerUnit: f.factorKgCo2PerUnit,
                      unit: f.unit, capturedAt: doc.createdAt }
  }, doc.extra || {});
}
```

mongosh — collections e contagem

```
================================================================================
  COLLECTIONS EXISTENTES
================================================================================
[
  'emission_factors',
  'governance_audits',
  'offset_projects',
  'participants',
  'simulations'
]

================================================================================
  CONTAGEM DE DOCUMENTOS
================================================================================
{
  emission_factors: 12,
  simulations: 14,
  participants: 12,
  offset_projects: 12,
  governance_audits: 12
}
TOTAL: 62 documentos
```

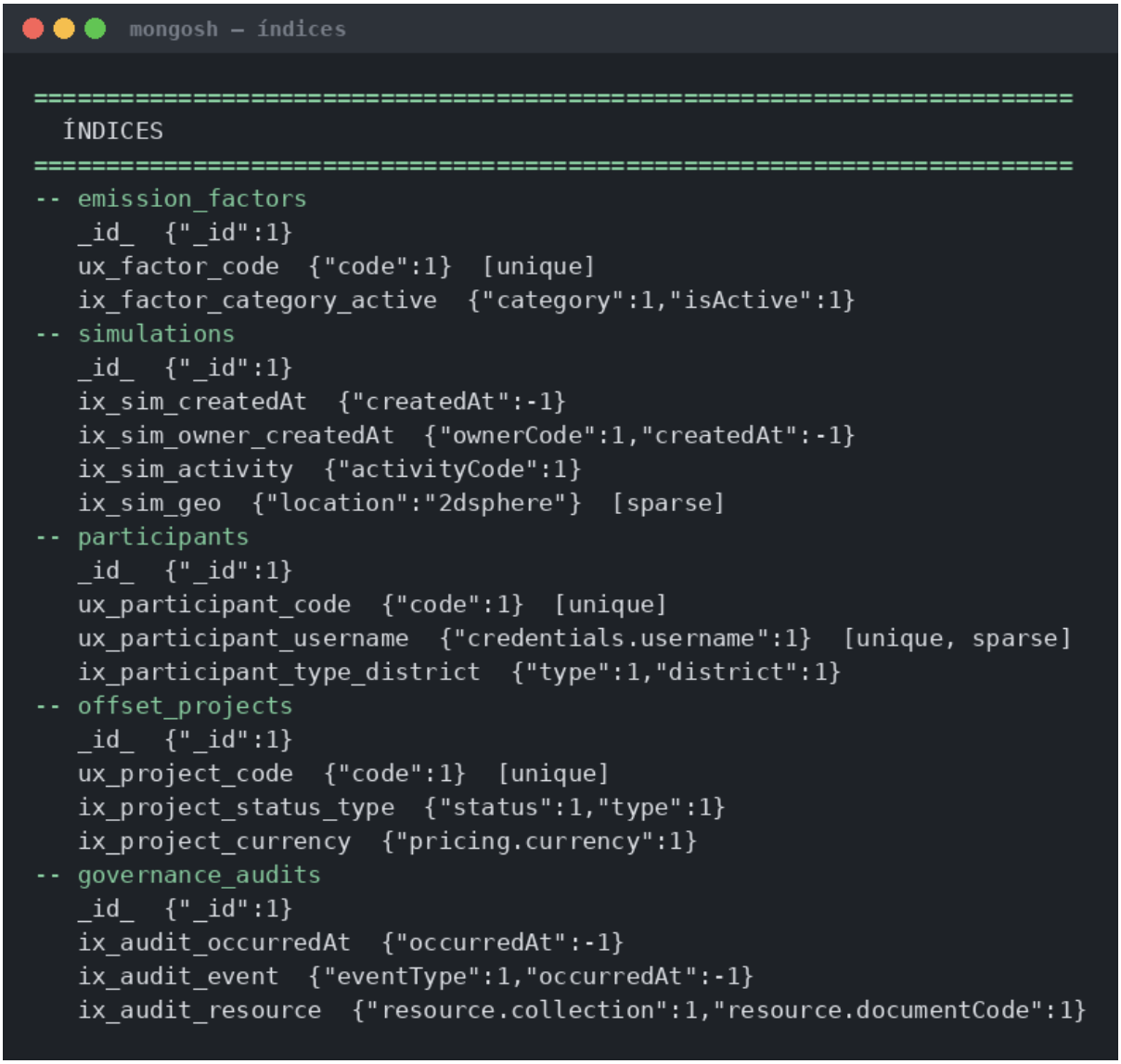

mongosh — índices
ÍNDICES
-- emission_factors
_id_ {"_id":1}
ux_factor_code {"code":1} [unique]
ix_factor_category_active {"category":1,"isActive":1}
-- simulations
_id_ {"_id":1}
ix_sim_createdAt {"createdAt":-1}
ix_sim_owner_createdAt {"ownerCode":1,"createdAt":-1}
ix_sim_activity {"activityCode":1}
ix_sim_geo {"location":"2dsphere"} [sparse]
-- participants
_id_ {"_id":1}
ux_participant_code {"code":1} [unique]
ux_participant_username {"credentials.username":1} [unique, sparse]
ix_participant_type_district {"type":1,"district":1}
-- offset_projects
_id_ {"_id":1}
ux_project_code {"code":1} [unique]
ix_project_status_type {"status":1,"type":1}
ix_project_currency {"pricing.currency":1}
-- governance_audits
_id_ {"_id":1}
ix_audit_occurredAt {"occurredAt":-1}
ix_audit_event {"eventType":1,"occurredAt":-1}
ix_audit_resource {"resource.collection":1,"resource.documentCode":1}



Figura 3 — Collections, contagem de documentos e índices (a, b, c). Fonte: elaborado pelo autor.

mongosh — documentos de emission_factors

```
======================================================================
  EMISSION_FACTORS  —  transporte com variants[] · energia com gridMix e monthlyFactors[] · documento mínimo
======================================================================

---- filtro: {"code":"CARRO"} ----
{
  _id: ObjectId('6aa0b29f98430455ceeba4e5'),
  code: 'CARRO',
  activity: 'Carro',
  factorKgCo2PerUnit: 0.21,
  unit: 'km',
  fieldLabel: 'Quilômetros percorridos',
  category: 'transporte',
  pillar: 'E',
  scope: 'escopo_3',
  isActive: true,
  source: {
    agency: 'GHG Protocol Brasil',
    year: 2025,
    methodology: 'tank-to-wheel'
  },
  variants: [
    {
      fuel: 'gasolina',
      factorKgCo2PerUnit: 0.21
    },
    {
      fuel: 'etanol',
      factorKgCo2PerUnit: 0.14
    },
    {
      fuel: 'diesel',
      factorKgCo2PerUnit: 0.25
    },
    {
      fuel: 'eletrico',
      factorKgCo2PerUnit: 0.05
    }
  ],
  updatedAt: ISODate('2026-09-09T01:13:03.162Z')
}

---- filtro: {"code":"ENERGIA_ELETRICA"} ----
{
  _id: ObjectId('6aa0b29f98430455ceeba4ee'),
  code: 'ENERGIA_ELETRICA',
  activity: 'Energia elétrica residencial',
  factorKgCo2PerUnit: 0.0385,
  unit: 'kWh',
  fieldLabel: 'kWh consumidos',
  category: 'energia',
  pillar: 'E',
  scope: 'escopo_2',
  isActive: true,
  source: {
    agency: 'MCTI - Fator médio SIN',
    year: 2025,
    methodology: 'grid average'
  },
  gridMix: {
    hidrica: 0.58,
    eolica: 0.14,
    solar: 0.09,
    termica: 0.16,
    nuclear: 0.03
  },
  monthlyFactors: [
    {
      month: 1,
      factor: 0.041
   [...]
```

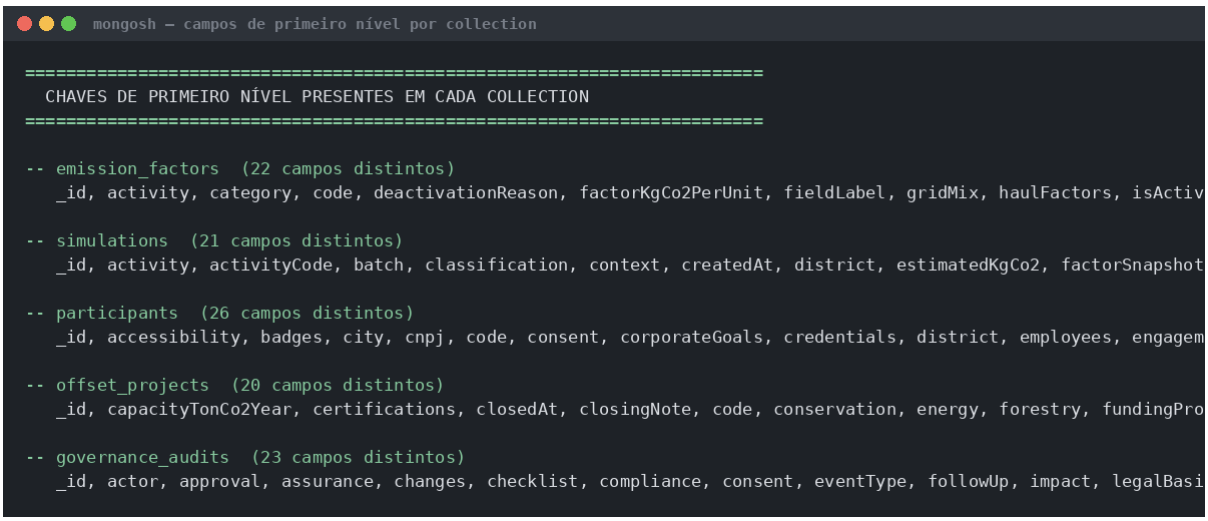


Figura 4 — Documentos de formatos distintos na mesma collection (a, b). Fonte: elaborado pelo autor.

5. Operações CRUD realizadas

Os scripts de mongo/crud executam operações de criação, leitura, atualização e remoção em cada uma das cinco collections. A forma de executá-los está descrita na seção 8.2.

5.1 CREATE — insertOne e insertMany

01-create.js — inserções nas 5 collections

```
// ---------- 1) emission_factors — insertOne ----------
db.emission_factors.insertOne({
  code: 'TREM_REGIONAL',
  activity: 'Trem regional',
  factorKgCo2PerUnit: 0.041,
  unit: 'km',
  fieldLabel: 'Quilômetros percorridos',
  category: 'transporte',
  pillar: 'E',
  scope: 'escopo_3',
  isActive: true,
  source: { agency: 'ANTT', year: 2026, methodology: 'tank-to-wheel' },
  updatedAt: new Date()
});

// ---------- 2) participants — insertMany com DUAS FORMAS diferentes ----------
db.participants.insertMany([
  { code: 'CID-008', type: 'cidadao', name: 'Helena Duarte',
    district: 'Vila Mariana', city: 'São Paulo', joinedAt: new Date(),
    engagement: { level: 'bronze', points: 0, streakDays: 0 },
    consent: { lgpdAccepted: true, shareAnonymizedData: true } },

  { code: 'ORG-004', type: 'organizacao', name: 'Bike Express Entregas',
    cnpj: '11.222.333/0001-44', sector: 'logistica', size: 'pequena', employees: 54,
    corporateGoals: [ { metric: 'tonCo2Ano', target: 9, baseline: 21 } ],
    socialPrograms: [ { name: 'Bolsa-manutenção para ciclistas', beneficiaries: 54 } ],
    consent: { lgpdAccepted: true, shareAnonymizedData: true } }
]);

// ---------- 3) simulations — insertOne com desnormalização do fator ----------
const factor = db.emission_factors.findOne({ code: 'TREM_REGIONAL' });
const value  = 260;
const kg     = Math.round(value * factor.factorKgCo2PerUnit * 100) / 100;

db.simulations.insertOne({
  activityCode: factor.code, activity: factor.activity,
  value: value, unit: factor.unit,
  estimatedKgCo2: kg,
  classification: kg <= 1 ? 'Baixo impacto' : (kg <= 10 ? 'Impacto moderado' : 'Alto impacto'),
  ownerCode: 'ORG-004', createdAt: new Date(), source: 'import',
  batch: { fileName: 'demo_crud.csv', rows: 1, importedBy: 'ANL-001' },
  factorSnapshot: { code: factor.code, factorKgCo2PerUnit: factor.factorKgCo2PerUnit,
                    unit: factor.unit, capturedAt: new Date() },
  location: { type: 'Point', coordinates: [-46.6390, -23.5890] }
});

// ---------- 4) offset_projects — insertOne ----------
db.offset_projects.insertOne({
  code: 'PRJ-013', name: 'Hortas Urbanas em Terrenos Ociosos',
  type: 'agricultura_urbana', status: 'piloto', capacityTonCo2Year: 42,
  pricing: [ { currency: 'BRL', pricePerTon: 135.0 } ],
```

```
  location: { type: 'Point', coordinates: [-46.6120, -23.5450], state: 'SP', city: 'São Paulo' },
  urbanFarming: { plots: 18, areaM2: 7400, cropsPerYear: 3 },
  socialImpact: { familiesSupplied: 260, jobsCreated: 14 },
  sdgTargets: [2, 11, 13], createdAt: new Date()
});

// ---------- 5) governance_audits — insertMany de DOIS TIPOS de evento ----------
db.governance_audits.insertMany([
  { eventType: 'factor_created', occurredAt: new Date(),
    actor: { participantCode: 'ANL-001', name: 'Ana Lúcia Ferreira', role: 'Analyst' },
    resource: { collection: 'emission_factors', documentCode: 'TREM_REGIONAL' },
    changes: { after: { factorKgCo2PerUnit: 0.041 },
               reason: 'Inclusão do modal ferroviário regional' },
    compliance: { framework: 'GHG Protocol', status: 'conforme' },
    demo: { script: '01-create' } },

  { eventType: 'project_registered', occurredAt: new Date(),
    actor: { participantCode: 'ANL-002', role: 'Analyst' },
    resource: { collection: 'offset_projects', documentCode: 'PRJ-013' },
    dueDiligence: { checklist: [ { item: 'Titularidade do terreno', status: 'ok' },
                                 { item: 'Certificação de crédito', status: 'pendente' } ],
                    riskLevel: 'baixo' },
    compliance: { framework: 'ISO 14064', status: 'parcial' },
    demo: { script: '01-create' } }
]);
```

mongosh — 01-create.js (emission_factors)

```
================================================================================
  CREATE 1/5 — emission_factors.insertOne()
================================================================================
{
  acknowledged: true,
  insertedId: ObjectId('6aa0b2a3755ce07f8e9309d3')
}
{
  _id: ObjectId('6aa0b2a3755ce07f8e9309d3'),
  code: 'TREM_REGIONAL',
  activity: 'Trem regional',
  factorKgCo2PerUnit: 0.041,
  unit: 'km',
  fieldLabel: 'Quilômetros percorridos',
  category: 'transporte',
  pillar: 'E',
  scope: 'escopo_3',
  isActive: true,
  source: {
    agency: 'ANTT',
    year: 2026,
    methodology: 'tank-to-wheel'
  },
  updatedAt: ISODate('2026-09-09T01:13:07.638Z')
}
```

mongosh — 01-create.js (participants)

```
================================================================================
  CREATE 2/5 — participants.insertMany()
================================================================================
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aa0b2a3755ce07f8e9309d4'),
    '1': ObjectId('6aa0b2a3755ce07f8e9309d5')
  }
}
{
  _id: ObjectId('6aa0b2a3755ce07f8e9309d4'),
  code: 'CID-008',
  type: 'cidadao',
  name: 'Helena Duarte'
}
{
  _id: ObjectId('6aa0b2a3755ce07f8e9309d5'),
  code: 'ORG-004',
  type: 'organizacao',
  name: 'Bike Express Entregas'
}
```

mongosh — 01-create.js (simulations)

```
================================================================================
  CREATE 3/5 — simulations.insertOne()
================================================================================
{
  acknowledged: true,
  insertedId: ObjectId('6aa0b2a3755ce07f8e9309d6')
}
{
  _id: ObjectId('6aa0b2a3755ce07f8e9309d6'),
  activityCode: 'TREM_REGIONAL',
  activity: 'Trem regional',
  value: 260,
  unit: 'km',
  estimatedKgCo2: 10.66,
  classification: 'Alto impacto',
  ownerCode: 'ORG-004',
  createdAt: ISODate('2026-09-09T01:13:07.656Z'),
  source: 'import',
  batch: {
    fileName: 'demo_crud.csv',
    rows: 1,
    importedBy: 'ANL-001'
  },
  factorSnapshot: {
    code: 'TREM_REGIONAL',
    factorKgCo2PerUnit: 0.041,
    unit: 'km',
    capturedAt: ISODate('2026-09-09T01:13:07.656Z')
  },
  location: {
    type: 'Point',
    coordinates: [
      -46.639,
      -23.589
    ]
  }
}
```

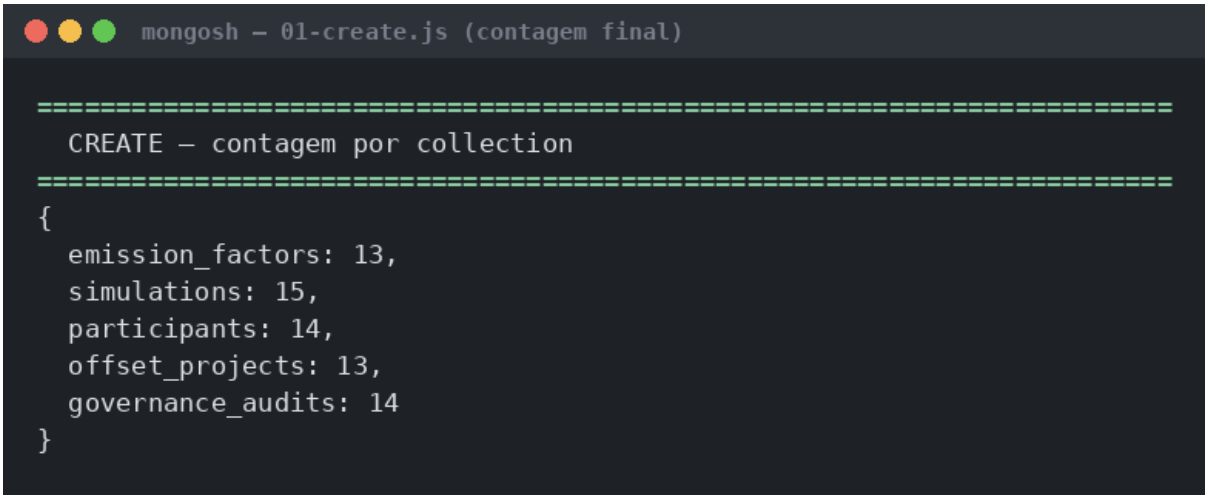


Figura 5 — Operações de CREATE (insertOne e insertMany) (a, b, c, d). Fonte: elaborado pelo autor.

5.2 READ — find, projeções, arrays, geoespacial e explain

02-read.js — leituras nas 5 collections

```
// ---------- 1) emission_factors ----------
db.emission_factors
  .find({ category: 'transporte', isActive: true },
        { _id: 0, code: 1, activity: 1, factorKgCo2PerUnit: 1, unit: 1 })
  .sort({ factorKgCo2PerUnit: -1 });

// consulta dentro de ARRAY DE SUBDOCUMENTOS (impossível sem JOIN no relacional)
db.emission_factors.findOne({ 'variants.fuel': 'etanol' }, { _id: 0, code: 1, variants: 1 });

db.emission_factors.distinct('category');

// quem POSSUI determinado bloco — a pergunta típica de esquema flexível
db.emission_factors.find({ regionalFactors: { $exists: true } }, { _id: 0, code: 1 });

// ---------- 2) simulations ----------
db.simulations
  .find({}, { _id: 0, activity: 1, value: 1, unit: 1, estimatedKgCo2: 1,
              classification: 1, ownerCode: 1, createdAt: 1 })
  .sort({ createdAt: -1 }).skip(0).limit(5);              // paginação

db.simulations.countDocuments();
db.simulations.countDocuments({ classification: 'Alto impacto' });

db.simulations.find({ createdAt: { $gte: ISODate('2026-08-20T00:00:00Z') },
                      estimatedKgCo2: { $gt: 10 } },
                    { _id: 0, activity: 1, estimatedKgCo2: 1, createdAt: 1 });

// só as leituras de sensores IoT
db.simulations.find({ 'iotDevice.sensorId': { $exists: true } },
                    { _id: 0, activity: 1, 'iotDevice.sensorId': 1,
                      'iotDevice.signalQuality': 1 });

// CONSULTA GEOESPACIAL — simulações a até 8 km da Praça da Sé
db.simulations.find({
  location: { $near: { $geometry: { type: 'Point', coordinates: [-46.6333, -23.5505] },
                       $maxDistance: 8000 } }
}, { _id: 0, activity: 1, district: 1, estimatedKgCo2: 1 });

// ---------- 3) participants ----------
```

```
db.participants.aggregate([ { $group: { _id: '$type', total: { $sum: 1 } } },
                            { $sort: { total: -1 } } ]);

// login: usa o índice único e esparso ux_participant_username
db.participants.findOne({ 'credentials.username': 'analista' },
                        { _id: 0, code: 1, name: 1, role: 1, 'credentials.username': 1 });

// cidadãos nível ouro com pelo menos uma meta atingida
db.participants.find({ type: 'cidadao', 'engagement.level': 'ouro',
                       goals: { $elemMatch: { achieved: true } } },
                     { _id: 0, code: 1, name: 1, engagement: 1 });

db.participants.find({ 'esgReporting.framework': 'GRI' },
                     { _id: 0, code: 1, name: 1, esgReporting: 1 });

// ---------- 4) offset_projects ----------
db.offset_projects.find({ status: 'ativo' },
                        { _id: 0, code: 1, name: 1, type: 1, capacityTonCo2Year: 1 })
                  .sort({ capacityTonCo2Year: -1 });

// preço dentro de array de subdocumentos
db.offset_projects.find({ pricing: { $elemMatch: { currency: 'USD',
                                                   pricePerTon: { $lte: 16 } } } },
                        { _id: 0, code: 1, name: 1, pricing: 1 });

db.offset_projects.find({ sdgTargets: 13 }, { _id: 0, code: 1, sdgTargets: 1 });
db.offset_projects.find({ 'socialImpact.jobsCreated': { $gte: 20 } },
                        { _id: 0, code: 1, name: 1, 'socialImpact.jobsCreated': 1 });

// ---------- 5) governance_audits ----------
db.governance_audits.aggregate([ { $group: { _id: '$eventType', total: { $sum: 1 } } },
                                 { $sort: { _id: 1 } } ]);

// histórico completo de um fator — rastreabilidade exigida em auditoria ESG
db.governance_audits.find({ 'resource.collection': 'emission_factors',
                            'resource.documentCode': 'CARRO' },
                          { _id: 0, eventType: 1, occurredAt: 1,
                            'actor.name': 1, changes: 1 })
                    .sort({ occurredAt: -1 });

db.governance_audits.find({ 'compliance.framework': 'LGPD' },
                          { _id: 0, eventType: 1, occurredAt: 1,
                            legalBasis: 1, 'compliance.status': 1 });

db.governance_audits.find({ 'compliance.status': { $ne: 'conforme' } },
                          { _id: 0, eventType: 1, 'resource.documentCode': 1, compliance: 1 });

// ---------- comprovação de uso de índice ----------
db.simulations.find({ ownerCode: 'ORG-001' })
              .sort({ createdAt: -1 })
              .explain('executionStats');
```

mongosh — 02-read.js (emission_factors)

```
================================================================================
  READ 1/5 — emission_factors: ativos de transporte, ordenados
================================================================================
{
  code: 'CARRO',
  activity: 'Carro',
  factorKgCo2PerUnit: 0.21,
  unit: 'km'
}
{
  code: 'AVIAO',
  activity: 'Avião',
  factorKgCo2PerUnit: 0.15,
  unit: 'km'
}
{
  code: 'MOTO',
  activity: 'Motocicleta',
  factorKgCo2PerUnit: 0.11,
  unit: 'km'
}
{
  code: 'ONIBUS',
  activity: 'Ônibus',
  factorKgCo2PerUnit: 0.08,
  unit: 'km'
}
{
  code: 'TREM_REGIONAL',
  activity: 'Trem regional',
  factorKgCo2PerUnit: 0.041,
  unit: 'km'
}
{
  code: 'METRO',
  activity: 'Metrô',
  factorKgCo2PerUnit: 0.03,
  unit: 'km'
}
{
  code: 'BICICLETA',
  activity: 'Bicicleta',
  factorKgCo2PerUnit: 0,
  unit: 'km'
}

  [...]
```

mongosh — 02-read.js (simulations: IoT e geoespacial)

```
-- somente leituras de sensores IoT (campo exclusivo desses documentos):
[
  {
    activity: 'Energia elétrica residencial',
    iotDevice: {
      sensorId: 'MED-SP-0471',
      signalQuality: 0.97
    }
  },
  {
    activity: 'Energia elétrica residencial',
    iotDevice: {
      sensorId: 'MED-SP-0471',
      signalQuality: 0.93
    }
  },
  {
    activity: 'Gás natural encanado',
    iotDevice: {
      sensorId: 'GAS-GRU-0092'
    }
  }
]

-- consulta GEOESPACIAL: simulações a até 8 km da Praça da Sé
[
  {
    activity: 'Energia elétrica residencial',
    estimatedKgCo2: 12.24
  },
  {
    activity: 'Energia elétrica residencial',
    estimatedKgCo2: 15.48
  },
  {
    activity: 'Trem regional',
    estimatedKgCo2: 10.66
  },
  {
    activity: 'Ônibus',
    estimatedKgCo2: 3.36,
    district: 'Butantã'
  }
]
```

mongosh — 02-read.js (participants)

```
================================================================================
  READ 3/5 — participants: polimorfismo e busca por credenciais
================================================================================
-- contagem por tipo:
[
  {
    _id: 'cidadao',
    total: 7
  },
  {
    _id: 'organizacao',
    total: 4
  },
  {
    _id: 'orgao_publico',
    total: 3
  }
]

-- login: busca do usuário "analista" (índice ux_participant_username):
{
  code: 'ANL-001',
  name: 'Ana Lúcia Ferreira',
  role: 'Analyst',
  credentials: {
    username: 'analista'
  }
}

-- cidadãos nível ouro com meta atingida ($elemMatch):
[
  {
    code: 'CID-007',
    name: 'Otávio Mendes',
    engagement: {
      level: 'ouro',
      points: 2015,
      streakDays: 33
    }
  },
  {
    code: 'ANL-001',
    name: 'Ana Lúcia Ferreira',
    engagement: {
      level: 'ouro',
      points: 2480,
      streakDays: 41
    }
  }
]
```

```
mongosh — 02-read.js (governance_audits)

-- histórico completo do fator CARRO (rastreabilidade):
[
  {
    eventType: 'factor_updated',
    occurredAt: ISODate('2026-07-02T14:03:00.000Z'),
    actor: {
      name: 'Ana Lúcia Ferreira'
    },
    changes: {
      before: {
        factorKgCo2PerUnit: 0.2
      },
      after: {
        factorKgCo2PerUnit: 0.21
      },
      reason: 'Atualização do inventário nacional 2025'
    }
  }
]
```

```
mongosh — 02-read.js (explain)

{
  stage: 'IXSCAN',
  indexName: [
    '"indexName":"ix_sim_owner_createdAt"',
    'ix_sim_owner_createdAt',
    index: 92,
    input: '{"stage":"FETCH","inputStage":{"stage":"IXSCAN","keyPattern":{"ownerCode":1,"createdAt":-1},"indexName"
    groups: undefined
  ],
  totalDocsExamined: 3,
  nReturned: 3,
  executionTimeMillis: 0
   [...]
```

Figura 6 — Operações de READ (find, projeções, consulta geoespacial) (a, b, c, d, e). Fonte: elaborado pelo autor.

5.3 UPDATE — $set, $inc, $push, $unset, upsert, arrayFilters e pipeline

03-update.js — atualizações nas 5 collections

```
// ---------- 1) emission_factors — $set + $push em array aninhado ----------
db.emission_factors.updateOne(
  { code: 'CARRO' },
  { $set:  { factorKgCo2PerUnit: 0.219, updatedAt: new Date(), 'source.year': 2026 },
    $push: { variants: { fuel: 'hibrido', factorKgCo2PerUnit: 0.12 } } }
);

// updateMany — marca todos os fatores de alimentação como revisados
db.emission_factors.updateMany(
  { category: 'alimentacao' },
  { $set: { review: { status: 'revisado', reviewedAt: new Date(), reviewer: 'ANL-001' } } }
);

// ---------- 2) simulations — updateMany + $unset ----------
```

```
db.simulations.updateMany(
  { estimatedKgCo2: { $gt: 500 } },
  { $set: { classification: 'Impacto crítico', reviewedAt: new Date() } }
);

// remove o flag de anomalia depois de revalidar o sensor
db.simulations.updateOne(
  { 'iotDevice.anomalyFlag': { $exists: true } },
  { $unset: { 'iotDevice.anomalyFlag': '' },
    $set:   { 'iotDevice.revalidatedAt': new Date() } }
);

// ---------- 3) participants — $inc, $push, operador posicional e UPSERT ----------
db.participants.updateOne(
  { code: 'CID-003' },
  { $inc:  { 'engagement.points': 150, 'engagement.streakDays': 1 },
    $set:  { 'engagement.level': 'ouro' },
    $push: { badges: 'meta-agosto' } }
);

// atualiza o item específico do array que casou com o filtro (operador posicional $)
db.participants.findOneAndUpdate(
  { code: 'CID-005', 'goals.metric': 'kgCo2Mes' },
  { $set: { 'goals.$.achieved': true, 'goals.$.achievedAt': new Date() } },
  { returnDocument: 'after', projection: { _id: 0, code: 1, goals: 1 } }
);

// UPSERT — cria o documento caso não exista
db.participants.updateOne(
  { code: 'ORG-005' },
  { $set: { type: 'organizacao', name: 'Padaria Bairro Verde',
            sector: 'alimentacao', size: 'micro', city: 'São Paulo',
            consent: { lgpdAccepted: true, shareAnonymizedData: true } },
    $setOnInsert: { joinedAt: new Date(), createdBy: 'upsert-demo' } },
  { upsert: true }
);

// ---------- 4) offset_projects — arrayFilters, $addToSet e UPDATE COM PIPELINE ----------
db.offset_projects.updateOne(
  { code: 'PRJ-001' },
  { $set: { 'pricing.$[el].pricePerTon': 97.5, updatedAt: new Date() } },
  { arrayFilters: [ { 'el.currency': 'BRL' } ] }
);

db.offset_projects.updateOne({ code: 'PRJ-001' },
                             { $addToSet: { certifications: 'Gold Standard' } });

// reajuste de 8% calculado dentro do próprio banco
db.offset_projects.updateMany(
  { status: 'em_captacao' },
  [ { $set: {
        pricing: { $map: { input: '$pricing', as: 'p',
                           in: { currency: '$$p.currency',
                                 pricePerTon: { $round: [ { $multiply: ['$$p.pricePerTon',
1.08] }, 2 ] } } } },
        lastRepricedAt: '$$NOW' } } ]
);

// ---------- 5) governance_audits — conclusão de revisão e arquivamento ----------
db.governance_audits.updateOne(
```

```
  { eventType: 'policy_review', 'policy.code': 'CTM-EL-01',
    'checklist.item': 'Plano de infraestrutura de recarga' },
  { $set: { 'checklist.$.status': 'ok',
            'checklist.$.completedAt': new Date(),
            'compliance.status': 'conforme',
            closedAt: new Date() } }
);

db.governance_audits.updateMany(
  { eventType: 'incident', 'resolution.status': 'resolvido',
    occurredAt: { $lt: ISODate('2026-08-01T00:00:00Z') } },
  { $set: { archived: true, archivedAt: new Date() } }
);
```

```
mongosh — 03-update.js (emission_factors)

================================================================================
  UPDATE 1/5 — emission_factors.updateOne() com $set e $push
================================================================================
-- ANTES:
{
  code: 'CARRO',
  factorKgCo2PerUnit: 0.21,
  variants: [
    {
      fuel: 'gasolina',
      factorKgCo2PerUnit: 0.21
    },
    {
      fuel: 'etanol',
      factorKgCo2PerUnit: 0.14
    },
    {
      fuel: 'diesel',
      factorKgCo2PerUnit: 0.25
    },
    {
      fuel: 'eletrico',
      factorKgCo2PerUnit: 0.05
    }
  ]
}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
-- DEPOIS:
{
  code: 'CARRO',
  factorKgCo2PerUnit: 0.219,
  source: {
    agency: 'GHG Protocol Brasil',
    year: 2026,
    methodology: 'tank-to-wheel'
  },
  variants: [
    {
      fuel: 'gasolina',
      factorKgCo2PerUnit: 0.21
    },
    {
      fuel: 'etanol',
      factorKgCo2PerUnit: 0.14
    [...]
```

mongosh — 03-update.js (participants)

```
================================================================================
  UPDATE 3/5 — participants: $inc (gamificação), $push e upsert
================================================================================
-- ANTES:
{
  code: 'CID-003',
  engagement: {
    level: 'prata',
    points: 940,
    streakDays: 12
  }
}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
-- DEPOIS:
{
  code: 'CID-003',
  engagement: {
    level: 'ouro',
    points: 1090,
    streakDays: 13
  },
  badges: [
    'meta-agosto'
  ]
}
```

mongosh — 03-update.js (upsert)

```
-- UPSERT: cria o participante se ainda não existir (não existe -> insere)
{
  acknowledged: true,
  insertedId: ObjectId('6aa0b2a5fd15705318bb30a6'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
{
  code: 'ORG-005',
  createdBy: 'upsert-demo',
  name: 'Padaria Bairro Verde'
}
```

mongosh — 03-update.js (offset_projects, pipeline)

```
-- UPDATE COM PIPELINE: aplica +8% em todo projeto em captação (campo calculado):
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
[
  {
    code: 'PRJ-006',
    pricing: [
      {
        currency: 'BRL',
        pricePerTon: 103.68
      }
    ]
  },
  {
    code: 'PRJ-010',
    pricing: [
      {
        currency: 'BRL',
        pricePerTon: 95.04
      },
      {
        currency: 'USD',
        pricePerTon: 18.9
      }
    ]
  },
  {
    code: 'PRJ-003',
    pricing: [
      {
        currency: 'BRL',
        pricePerTon: 88.56
      }
    ]
  }
   [...]
```



Figura 7 — Operações de UPDATE (antes e depois) (a, b, c, d, e). Fonte: elaborado pelo autor.

5.4 DELETE — deleteOne, deleteMany, findOneAndDelete e soft delete

04-delete.js — remoções nas 5 collections

```
// ---------- 1) simulations — deleteMany por filtro de lote ----------
db.simulations.deleteMany({ 'batch.fileName': 'demo_crud.csv' });

// ---------- 2) participants — deleteOne e findOneAndDelete ----------
db.participants.deleteOne({ code: 'CID-008' });

// findOneAndDelete devolve o documento removido — útil como log de auditoria
db.participants.findOneAndDelete({ code: 'ORG-004' },
                                 { projection: { _id: 0, code: 1, name: 1, type: 1 } });

db.participants.deleteOne({ code: 'ORG-005', createdBy: 'upsert-demo' });

// ---------- 3) offset_projects — deleteOne ----------
db.offset_projects.deleteOne({ code: 'PRJ-013' });

// ---------- 4) emission_factors — SOFT DELETE e depois delete físico ----------
// soft delete: preserva o histórico das simulações que usaram este fator
db.emission_factors.updateOne(
  { code: 'TREM_REGIONAL' },
  { $set: { isActive: false, deactivatedAt: new Date(),
            deactivationReason: 'demonstração de soft delete' } }
);

db.emission_factors.deleteOne({ code: 'TREM_REGIONAL' });

// ---------- 5) governance_audits — remoção apenas dos registros de demonstração ----------
// OBS.: em produção esta collection é APPEND-ONLY; a remoção existe
//       somente para tornar o roteiro de CRUD repetível.
db.governance_audits.deleteMany({ 'demo.script': '01-create' });

// contagem final das 5 collections
printjson({
  emission_factors:  db.emission_factors.countDocuments(),
  simulations:       db.simulations.countDocuments(),
  participants:      db.participants.countDocuments(),
  offset_projects:   db.offset_projects.countDocuments(),
  governance_audits: db.governance_audits.countDocuments()
});
```

A distinção entre soft delete e delete físico é uma decisão de governança, não de tecnologia: um fator de emissão nunca deve sumir enquanto existirem simulações que o utilizaram, sob pena de quebrar a rastreabilidade do inventário. Por isso o fator é desativado (isActive: false) com motivo e data, e só é removido fisicamente quando se trata de dado de demonstração.

mongosh — 04-delete.js (antes)

```
================================================================================
  DELETE — contagem ANTES
================================================================================
{
  emission_factors: 13,
  simulations: 15,
  participants: 15,
  offset_projects: 13,
  governance_audits: 14
}
```

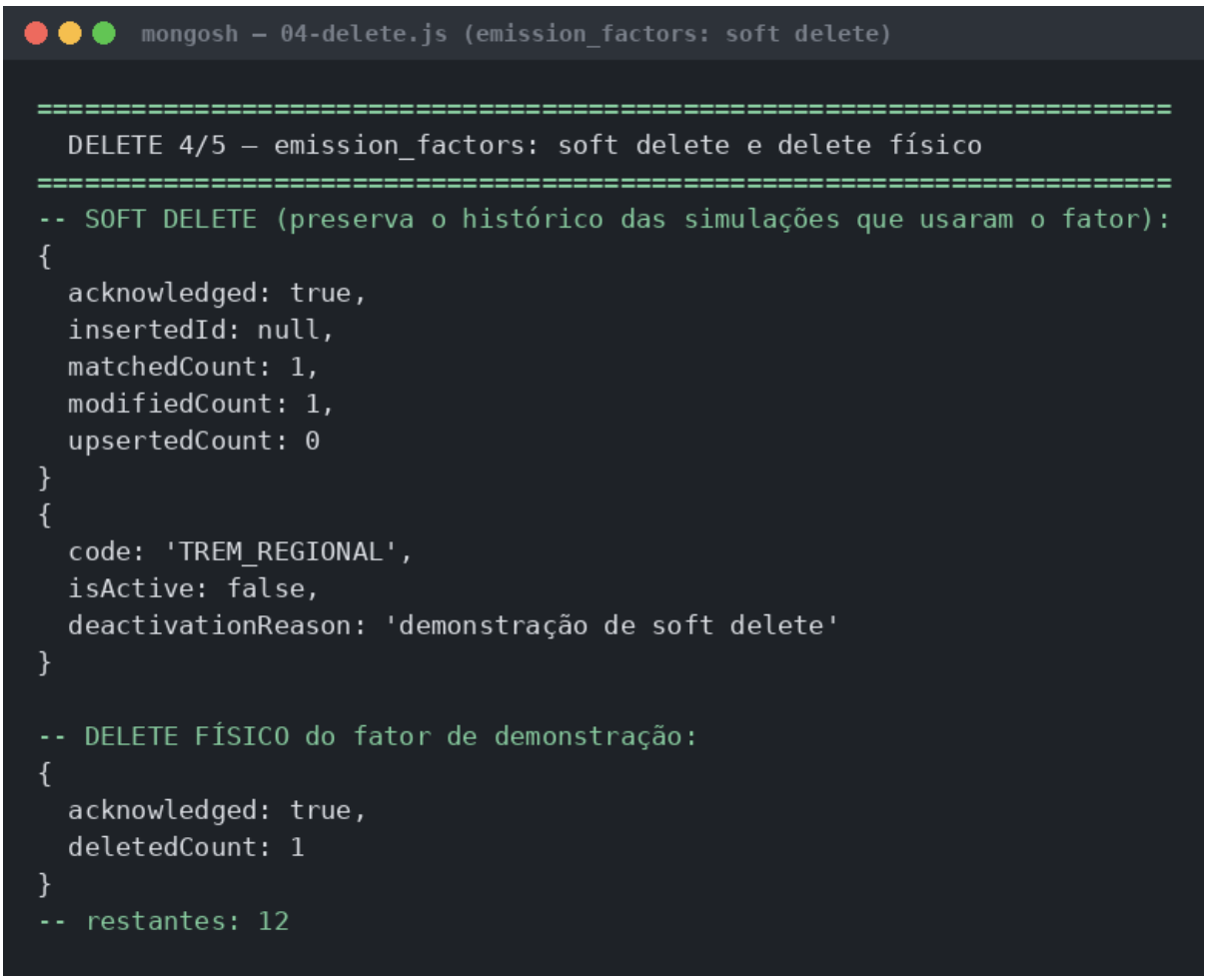
mongosh — 04-delete.js (simulations e participants)

```
================================================================================
  DELETE 1/5 — simulations.deleteMany({ "batch.fileName": "demo_crud.csv" })
================================================================================
-- alvo: 1 documento(s)
{
  acknowledged: true,
  deletedCount: 1
}
-- restantes: 14

================================================================================
  DELETE 2/5 — participants.deleteOne() e findOneAndDelete()
================================================================================
{
  acknowledged: true,
  deletedCount: 1
}

-- findOneAndDelete devolve o documento removido (útil para log de auditoria):
{
  code: 'ORG-004',
  type: 'organizacao',
  name: 'Bike Express Entregas'
}

-- remoção do participante criado por upsert:
{
  acknowledged: true,
  deletedCount: 1
}
-- restantes: 12
```



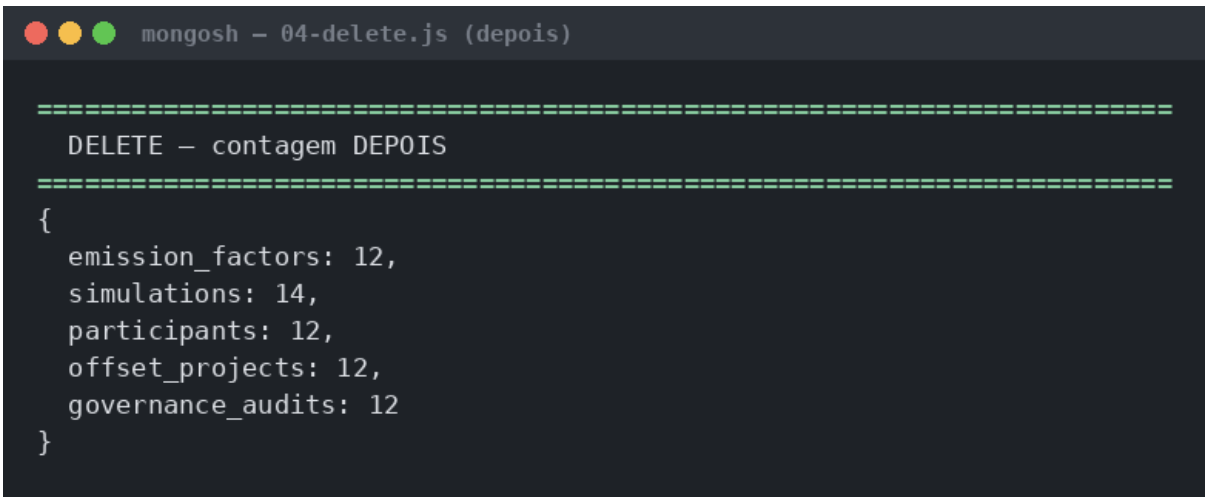


Figura 8 — Operações de DELETE (contagem antes e depois) (a, b, c, d). Fonte: elaborado pelo autor.

6. Aggregation Framework — o que era JOIN e GROUP BY

O script mongo/crud/05-aggregations.js reproduz, em pipelines, as consultas analíticas que no modelo relacional exigiriam JOIN e GROUP BY. O dashboard, por exemplo, é resolvido em um único $facet, que devolve totais, ranking por atividade, tendência diária e distribuição por origem em uma só ida ao banco.

05-aggregations.js (trechos)

```
// ---------- Dashboard ESG em UM ÚNICO $facet ----------
db.simulations.aggregate([
  { $facet: {
      totais: [
        { $group: { _id: null,
                    totalSimulacoes: { $sum: 1 },
                    totalKgCo2: { $sum: '$estimatedKgCo2' },
                    mediaKgCo2: { $avg: '$estimatedKgCo2' } } }
      ],
      rankingPorAtividade: [
        { $group: { _id: '$activity', simulacoes: { $sum: 1 },
                    totalKgCo2: { $sum: '$estimatedKgCo2' } } },
        { $sort: { totalKgCo2: -1 } }, { $limit: 5 }
      ],
      tendenciaDiaria: [
        { $group: { _id: { $dateToString: { format: '%Y-%m-%d', date: '$createdAt' } },
                    totalKgCo2: { $sum: '$estimatedKgCo2' } } },
        { $sort: { _id: 1 } }
      ],
      porOrigem: [
        { $group: { _id: '$source', total: { $sum: 1 } } },
        { $sort: { total: -1 } }
      ]
  } }
]);

// ---------- $lookup — o "JOIN" do MongoDB, usado só onde faz sentido ----------
db.simulations.aggregate([
  { $group: { _id: '$ownerCode', simulacoes: { $sum: 1 },
              totalKgCo2: { $sum: '$estimatedKgCo2' } } },
  { $lookup: { from: 'participants', localField: '_id',
               foreignField: 'code', as: 'participante' } },
  { $unwind: { path: '$participante', preserveNullAndEmptyArrays: true } },
  { $project: { _id: 0, codigo: '$_id', nome: '$participante.name',
                tipo: '$participante.type', simulacoes: 1,
                totalKgCo2: { $round: ['$totalKgCo2', 2] } } },
  { $sort: { totalKgCo2: -1 } }
]);

// ---------- Cotação de compensação nos projetos reais ----------
db.offset_projects.aggregate([
  { $match: { status: 'ativo' } },
  { $unwind: '$pricing' },
  { $match: { 'pricing.currency': 'BRL' } },
  { $project: { _id: 0, codigo: '$code', projeto: '$name', tipo: '$type',
                precoPorTonBRL: '$pricing.pricePerTon',
                custoEstimadoBRL: { $round: [ { $multiply: [totalTon, '$pricing.pricePerTon'] },
2 ] },
                arvoresEquivalentes: { $ceil: { $divide: [totalKg, 21] } } } },
  { $sort: { precoPorTonBRL: 1 } }, { $limit: 3 }
]);
```

```
// ---------- Governança: conformidade por framework (pilar G) ----------
db.governance_audits.aggregate([
  { $group: { _id: '$compliance.framework', eventos: { $sum: 1 },
              conformes: { $sum: { $cond: [ { $eq: ['$compliance.status', 'conforme'] }, 1, 0 ] }
} } },
  { $project: { _id: 0, framework: '$_id', eventos: 1, conformes: 1,
                percentualConformidade: {
                  $round: [ { $multiply: [ { $divide: ['$conformes', '$eventos'] }, 100 ] },
1 ] } } },
  { $sort: { framework: 1 } }
]);

// ---------- Social: engajamento por bairro (pilar S) ----------
db.participants.aggregate([
  { $match: { type: 'cidadao' } },
  { $group: { _id: '$district', cidadaos: { $sum: 1 },
              pontosMedios: { $avg: '$engagement.points' },
              nivelOuro: { $sum: { $cond: [ { $eq: ['$engagement.level', 'ouro'] }, 1,
0 ] } } } },
  { $sort: { pontosMedios: -1 } }
]);
```

mongosh — 05-aggregations.js ($facet)

```
================================================================================
  AGG 1 — Dashboard ESG ($facet)
================================================================================
[
  {
    totais: [
      {
        totalSimulacoes: 14,
        totalKgCo2: 1606.82,
        mediaKgCo2: 114.77
      }
    ],
    rankingPorAtividade: [
      {
        simulacoes: 2,
        atividade: 'Ônibus',
        totalKgCo2: 691.36
      },
      {
        simulacoes: 2,
        atividade: 'Carro',
        totalKgCo2: 474.6
      },
      {
        simulacoes: 1,
        atividade: 'Avião',
        totalKgCo2: 237
      },
      {
        simulacoes: 1,
        atividade: 'Gás natural encanado',
        totalKgCo2: 94.94
      },
      {
        simulacoes: 1,
        atividade: 'Motocicleta',
        totalKgCo2: 34.1
      }
    ],
```

mongosh — 05-aggregations.js ($lookup)

```
================================================================================
  AGG 2 — Emissão por participante ($lookup + $group)
================================================================================
[
  {
    simulacoes: 1,
    codigo: 'GOV-001',
    nome: 'Companhia de Transporte Metropolitano',
    tipo: 'orgao_publico',
    totalKgCo2: 688
  },
  {
    simulacoes: 3,
    codigo: 'ORG-001',
    nome: 'TechVerde Sistemas Ltda',
    tipo: 'organizacao',
    totalKgCo2: 477.12
  },
  {
    simulacoes: 1,
    codigo: 'ANL-001',
    nome: 'Ana Lúcia Ferreira',
    tipo: 'cidadao',
    totalKgCo2: 237
  },
  {
    simulacoes: 1,
    codigo: 'ORG-003',
    nome: 'Rede Alimenta Bairro',
    tipo: 'organizacao',
    totalKgCo2: 94.94
  },
  {
    simulacoes: 2,
    codigo: 'CID-004',
    nome: 'Beatriz Nunes',
    [...]
```

mongosh — 05-aggregations.js (recomendações e cotação)

```
======================================================================
  AGG 3 — Recomendações: economia projetada por troca de modal
======================================================================
[
  {
    atividade: 'Ônibus',
    alternativa: 'Bicicleta',
    economiaKgCo2: 691.36,
    atualKgCo2: 691.36,
    projetadoKgCo2: 0
  },
  {
    atividade: 'Carro',
    alternativa: 'Ônibus',
    economiaKgCo2: 293.8,
    atualKgCo2: 474.6,
    projetadoKgCo2: 180.8
  },
  {
    atividade: 'Avião',
    alternativa: 'Ônibus',
    economiaKgCo2: 110.6,
    atualKgCo2: 237,
    projetadoKgCo2: 126.4
  },
  {
    atividade: 'Motocicleta',
    alternativa: 'Ônibus',
    economiaKgCo2: 9.3,
    atualKgCo2: 34.1,
    projetadoKgCo2: 24.8
  }
]

======================================================================
  AGG 4 — Cotação de compensação em BRL (top 3 projetos mais baratos)
======================================================================
-- emissão total da base: 1606.82 kg CO2  (1.6068 t)
[
  {
    codigo: 'PRJ-008',
    projeto: 'Captura de Metano em Aterro Sanitário',
    tipo: 'captura_metano',
    precoPorTonBRL: 58,
    custoEstimadoBRL: 93.2,
    arvoresEquivalentes: 77
  },
  {
    codigo: 'PRJ-005',
    projeto: 'Parque Eólico Litoral Norte',
    tipo: 'energia_eolica',
    precoPorTonBRL: 68,
    custoEstimadoBRL: 109.26,
    arvoresEquivalentes: 77
  },
  {
    [...]
```

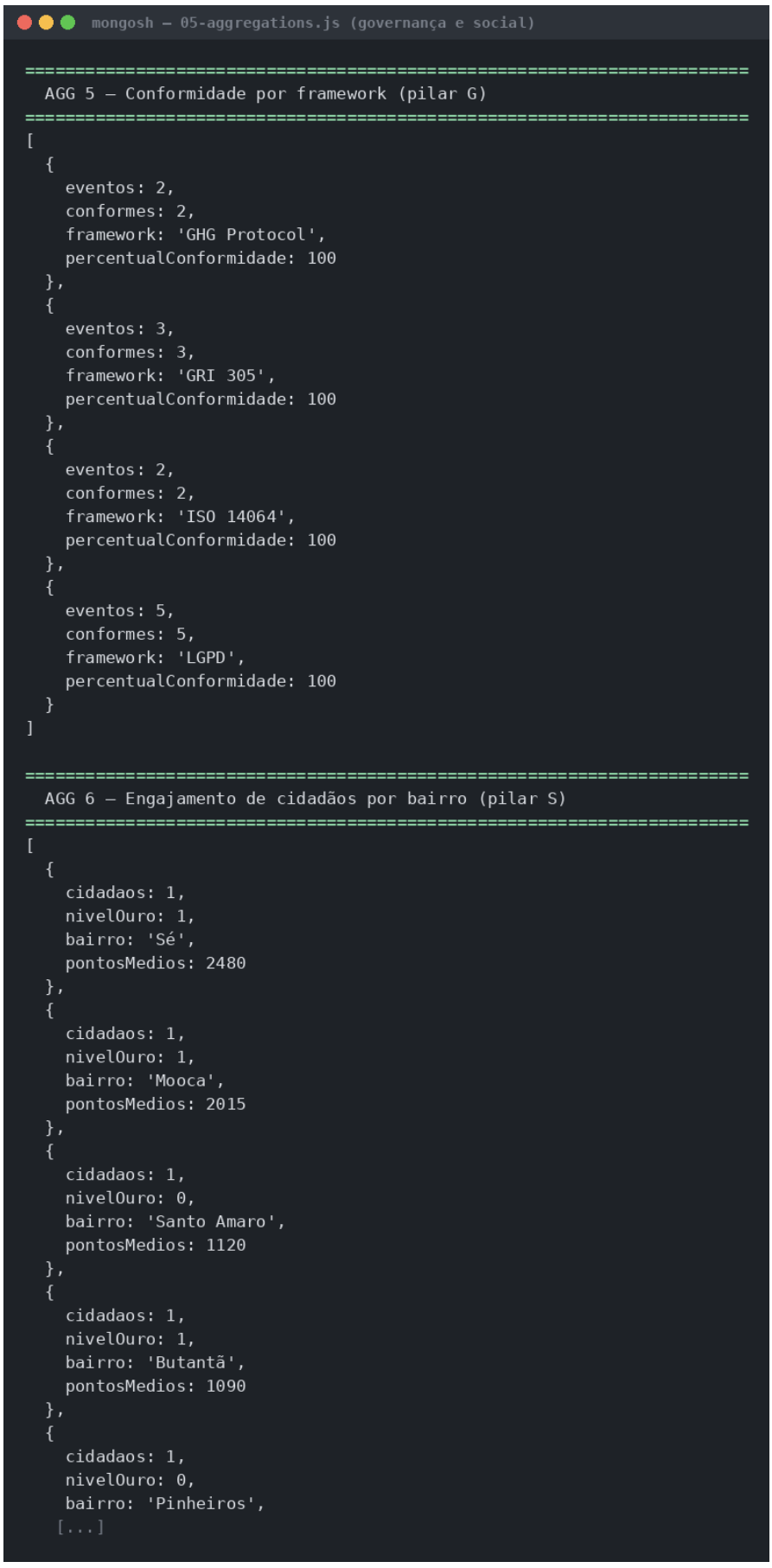


Figura 9 — Consultas de agregação (a, b, c, d). Fonte: elaborado pelo autor.

7. Backend .NET 8 migrado

O backend foi migrado preservando as interfaces de serviço e os ViewModels do projeto anterior — ISimulationService, IAnalyticsService, IRecommendationService, IOffsetService e IAuthService continuam com a mesma assinatura. O que mudou foi inteiramente a camada de dados.

7.1 Correspondência entre o modelo relacional e o de documentos

Projeto anterior (SQL Server + EF Core)	Projeto atual (MongoDB)
3 tabelas (EmissionFactors, Simulations, Users)	5 collections cobrindo E, S e G
AppDbContext / DbSet<T>	MongoContext / IMongoCollection<T>
Migrations + Database.Migrate()	MongoBootstrapper (criação de índices) + scripts de mongo/init
DbSeeder.SeedDefaultUser	03-seed-participants.js (mesmo hash PBKDF2-SHA256)
Tabela Users separada	Subdocumento credentials embutido no participante
Consultas LINQ com GroupBy	Aggregation Pipeline ($facet, $group, $lookup)
Novo atributo ⇒ nova coluna (migration)	Novo campo no documento
Classe C# precisa declarar todos os campos	[BsonExtraElements] lê e regrava campos não declarados

7.2 MongoContext — o substituto do AppDbContext

src/ESGImpact.Api/Data/MongoContext.cs

```
public class MongoContext
{
    public IMongoDatabase Database { get; }

    public MongoContext(IOptions<MongoSettings> options)
    {
        var settings = options.Value;
        var client = new MongoClient(settings.ConnectionString);
        Database = client.GetDatabase(settings.Database);
    }

    public IMongoCollection<EmissionFactor>  EmissionFactors  =>
Database.GetCollection<EmissionFactor>("emission_factors");
    public IMongoCollection<Simulation>      Simulations      =>
Database.GetCollection<Simulation>("simulations");
    public IMongoCollection<Participant>     Participants     =>
Database.GetCollection<Participant>("participants");
    public IMongoCollection<OffsetProject>   OffsetProjects   =>
Database.GetCollection<OffsetProject>("offset_projects");
    public IMongoCollection<GovernanceAudit> GovernanceAudits =>
Database.GetCollection<GovernanceAudit>("governance_audits");

    // acesso "cru", usado pelos endpoints genéricos de CRUD
    public IMongoCollection<BsonDocument> Raw(string collectionName)
        => Database.GetCollection<BsonDocument>(collectionName);
}
```

7.3 Como uma API tipada convive com esquema flexível

Este é o ponto técnico mais delicado da migração. C# é estaticamente tipado; o MongoDB não impõe forma aos documentos. A conciliação usa dois mecanismos:

- [BsonExtraElements] — a entidade declara apenas o núcleo estável; todo campo presente no documento e não declarado na classe é lido para um BsonDocument e regravado sem perda.
- Endpoints genéricos sobre BsonDocument — o corpo da requisição é convertido direto em documento BSON, de modo que um POST pode enviar campos que nenhuma classe declara.

src/ESGImpact.Api/Models/Entities.cs (trecho)

```
public class EmissionFactor
{
    [BsonId, BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }

    public string Code { get; set; } = string.Empty;
    public string Activity { get; set; } = string.Empty;
    public double FactorKgCo2PerUnit { get; set; }
    public string Unit { get; set; } = string.Empty;
    public bool   IsActive { get; set; } = true;

    // variants, gridMix, seasonality, regionalFactors, monthlyFactors...
    // entram aqui sem nenhuma classe e sem nenhuma migração
    [BsonExtraElements]
    public BsonDocument Extra { get; set; } = new();
}
```

Program.cs — convenções de serialização

```
var conventions = new ConventionPack
{
    new CamelCaseElementNameConvention(),   // FactorKgCo2PerUnit  <->  factorKgCo2PerUnit
    new IgnoreIfNullConvention(true),       // não grava campos nulos: documentos enxutos
    new EnumRepresentationConvention(BsonType.String)
};
ConventionRegistry.Register("esgimpact-conventions", conventions, _ => true);
```

7.4 Endpoints

CRUD completo das 5 collections (rotas: emission-factors, simulation-documents, participants, offset-projects, governance-audits):

Método	Rota	Operação MongoDB
GET	/api/{collection}	find + paginação + filtro JSON livre
GET	/api/{collection}/{id}	findOne — aceita o ObjectId ou o code de negócio
POST	/api/{collection}	insertOne
POST	/api/{collection}/bulk	insertMany
PATCH	/api/{collection}/{id}	updateOne com $set
PUT	/api/{collection}/{id}	replaceOne
DELETE	/api/{collection}/{id}	deleteOne

Endpoints de domínio migrados:

Método	Rota	Descrição	JWT
POST	/api/auth/login	Emite o JWT — credenciais embutidas em participants	—
POST	/api/simulations	Calcula, classifica e persiste com factorSnapshot	—
GET	/api/simulations	Histórico paginado	—
GET	/api/analytics/dashboard	Dashboard ESG em um único $facet	sim
GET	/api/recommendations	Economia de CO_2 quantificada por troca de modal	—
POST	/api/offset/quote	Cotação nos projetos reais da collection	sim
GET	/api/health e /api/collections/stats	Diagnóstico da conexão e contagens	—

Toda escrita relevante feita pela API gera automaticamente um documento em governance_audits (AuditService), com o ator extraído das claims do JWT.

Os endpoints da API são opcionais para esta entrega e podem ser explorados pelo Swagger em http://localhost:8080/swagger após "docker compose --profile api up -d --build". O arquivo src/ESGImpact.Api/ESGImpact.Api.http traz a sequência completa de chamadas.

8. Como executar o projeto

8.1 Subir a infraestrutura

Com o Docker instalado, a partir da raiz do projeto:

```
cp .env.example .env        # opcional: ajusta portas e senhas
docker compose up -d        # sobe MongoDB e Mongo Express
docker compose ps           # confirma que os containers estão saudáveis
```

Na primeira subida (volume vazio) o container executa automaticamente os scripts de mongo/init: cria as cinco collections, os índices e insere os 62 documentos da carga inicial. Para recarregar tudo do zero:

```
docker compose down -v && docker compose up -d
```

A interface web Mongo Express fica disponível em http://localhost:8081 para inspecionar as collections e os documentos.

8.2 Executar os scripts de CRUD

Os scripts de mongo/crud ficam montados no container em /scripts/crud e são executados com o mongosh. Cada comando abaixo roda um script e imprime o resultado das operações no terminal:

```
# inventário: collections, contagem de documentos, índices e validador
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/00-inventario.js

# amostra de documentos com formatos diferentes em cada collection
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/00b-amostra-
documentos.js

# CREATE — insertOne e insertMany nas 5 collections
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/01-create.js

# READ — find, projeções, arrays, consulta geoespacial e explain
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/02-read.js

# UPDATE — $set, $inc, $push, $unset, upsert, arrayFilters e pipeline
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/03-update.js

# DELETE — deleteOne, deleteMany, findOneAndDelete e soft delete
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/04-delete.js

# Aggregation Framework — dashboard, $lookup, cotação e indicadores ESG
docker compose exec -T esgimpact-mongo mongosh --quiet \
  -u esgadmin -p 'Esg@Mongo2026' --authenticationDatabase admin /scripts/crud/05-aggregations.js
```

A ordem recomendada é a numérica: o script de CREATE insere documentos de demonstração que os scripts de READ e UPDATE utilizam, e o de DELETE os remove, devolvendo a base ao estado da carga inicial. Os scripts podem ser executados quantas vezes forem necessárias.

Os mesmos comandos também podem ser digitados diretamente no shell interativo:

```
docker compose exec esgimpact-mongo mongosh -u esgadmin -p 'Esg@Mongo2026' \
  --authenticationDatabase admin esgimpact
```

```
esgimpact> show collections
esgimpact> db.emission_factors.countDocuments()
esgimpact> db.participants.find({ type: 'organizacao' }, { _id: 0, code: 1, name: 1 })
```

8.3 Executar a API .NET

```
# opção A — pelo Visual Studio ou pela CLI, apontando para localhost:27017
dotnet run --project src/ESGImpact.Api

# opção B — em container, junto com o banco
docker compose --profile api up -d --build

# Swagger: http://localhost:8080/swagger
# Login:   POST /api/auth/login  { "username": "analista", "password": "Esg@2026" }
```

O arquivo src/ESGImpact.Api/ESGImpact.Api.http contém a sequência completa de chamadas HTTP (login, CRUD das cinco collections e endpoints analíticos).

8.4 Credenciais do ambiente

Onde	Usuário	Senha
MongoDB (root)	esgadmin	Esg@Mongo2026
Mongo Express	admin	esgadmin
API — analista	analista	Esg@2026
API — gestor público	gestor.esg	Gov@2026
API — cidadão	cidadao.ml	Cid@2026

Valores de demonstração, versionados de propósito para o trabalho acadêmico. Em produção, as credenciais viriam de um arquivo .env fora do controle de versão ou de um gerenciador de segredos.

8.5 Arquivos entregues

Arquivo	Conteúdo
mongo/init/*.js	Criação das collections, validador $jsonSchema, índices e carga inicial (62 documentos)
mongo/crud/*.js	Roteiro completo de CRUD e de agregações
docker-compose.yml	Infraestrutura: MongoDB, Mongo Express, API, rede, volumes e healthcheck
src/ESGImpact.Api/	Backend .NET 8 migrado para o driver do MongoDB
src/ESGImpact.Api/ ESGImpact.Api.http	Roteiro de chamadas HTTP de todos os endpoints

9. Conclusão

A migração do ESGImpact para MongoDB permitiu representar o domínio ESG em cinco collections que cobrem os pilares ambiental, social e de governança, com operações completas de CRUD em cada uma delas e mais de dez documentos por collection.

- O esquema flexível acomoda fatores de emissão, projetos de compensação e eventos de governança com formas diferentes na mesma collection, sem colunas nulas nem tabelas auxiliares.

- Os documentos polimórficos de participants reúnem cidadãos, organizações e órgãos públicos em um único cadastro, com credenciais embutidas e índice esparso.
- A desnormalização controlada em simulations.factorSnapshot preserva o histórico de emissões independentemente de alterações posteriores nos fatores.
- O Aggregation Framework concentra em pipelines as consultas analíticas — dashboard, recomendações, cotação de compensação, conformidade e engajamento.
- Os índices geoespaciais permitem analisar emissões por região da cidade, requisito central de uma aplicação para cidades inteligentes.

O conjunto — scripts de criação e carga, roteiro de CRUD, infraestrutura em Docker e backend .NET migrado — forma uma base coerente para evoluir a aplicação conforme novos sensores, metodologias de cálculo e exigências regulatórias forem incorporados.